see here at the end the story is alice make contract and publish on ethrium blockchain or any other blockchain after that bob is there and he can mint nft threw alice dapp after that in the bob account nft is showed now bob go to any nft marketplace suppose opensea and there he put his nft on sell am i right?

✅ **Yes, you are absolutely right!** Here's a step-by-step breakdown of the full process:

📌 Step-by-Step NFT Minting & Selling Process

🔹 Step 1: Alice Creates & Deploys the NFT Smart Contract

- Alice writes a **smart contract** that allows minting NFTs.
- She **deploys the contract** on **Ethereum (or any other blockchain like Polygon, Solana, etc.)**.
- This contract manages **NFT ownership, minting rules, and metadata storage**.

🔹 Step 2: Bob Mints an NFT from Alice's DApp

- Bob visits **Alice's DApp (a website or app that interacts with her smart contract)**.
- He **connects his wallet (MetaMask, Trust Wallet, etc.)**.
- Bob **mints an NFT** by **sending ETH** (or another token) to Alice's contract.
- ✅ The contract **creates a new NFT** and assigns it to Bob's wallet.

🚀 **Now, the NFT exists on the blockchain inside Alice's contract.**

🔹 Step 3: Bob's NFT Appears in His Wallet & OpenSea

- Bob **checks his wallet** (some wallets support NFTs natively).
- **OpenSea (or other marketplaces) automatically detects the NFT!**
  - OpenSea **reads blockchain data** and shows NFTs owned by Bob.
  - Bob **doesn't need to upload anything**—it's already on OpenSea.

🔹 Step 4: Bob Lists His NFT for Sale on OpenSea

- Bob goes to **OpenSea** → Connects his wallet.
- He **finds his NFT** in the "Owned NFTs" section.
- He clicks "**Sell**", sets the price (e.g., 0.1 ETH).
- Bob signs a **transaction to approve selling** (this allows OpenSea to handle the sale).
- 🚀 Now, Bob's NFT is **listed on OpenSea for sale**!

🔹 Step 5: Someone (David) Buys Bob's NFT

- David **sees Bob's NFT on OpenSea** and buys it.
- ✅ **Smart contract transfers ownership** from Bob to David.
- ✅ **Bob receives 0.09 ETH** (after marketplace fees & royalties).
- ✅ **Alice (creator) receives royalties** (e.g., 5% of the sale).

🚀 **Now, David owns the NFT, and OpenSea updates automatically!**

🚀 Final Answer to Your Question:

✅ Yes!

- Alice creates the contract & deploys it on a blockchain.
- Bob mints an NFT through Alice's DApp.
- The NFT appears in Bob's wallet & OpenSea (because OpenSea reads blockchain data).
- Bob lists it for sale on OpenSea.
- David buys it, and ownership transfers via blockchain.

📌 OpenSea is just a marketplace—it doesn't store NFTs, it just reads the blockchain!

ERC721

ERC-721 is a standard for creating non-fungible tokens (NFTs) on the Ethereum blockchain.

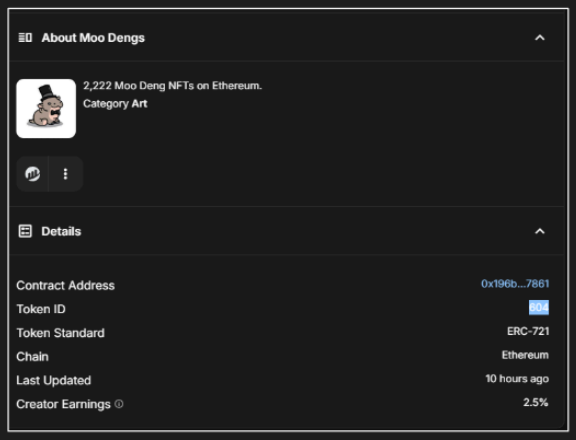
ERC-721 defines core functions for managing unique tokens:

1. **balanceOf**: Returns the number of tokens owned by a specific address.
2. **ownerOf**: Identifies the owner of a particular token.
3. **transferFrom**: Transfers token ownership from one address to another.
4. **approve**: Grants permission for another address to transfer a token.
5. **setApprovalForAll**: Authorizes or removes an operator's rights over all tokens.
6. **getApproved**: Checks if an address is approved to transfer a specific token.
7. **isApprovedForAll**: Verifies if an operator can manage all of an owner's tokens.

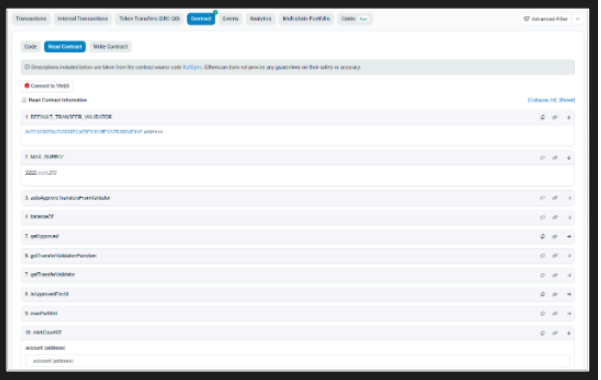
ERC20 vs ERC721

Property	ERC20	ERC721
Fungibility	Tokens are fungible up to the decimal places defined in the contract.	Tokens are non-fungible. Each token itself represents 1 token, which cannot be sub-divided.
Ownership	Token ownership is not directly linked to an account, and only the token balances are tracked in the contract.	Each token's ownership is linked to an individual account address.
Uniqueness	One token is similar in economic value to another token from the same token contract. There is no difference in each token.	Each token is different than each other token present in the same contract and may have different economic value.
Example	Because of this divisible and identical property, the ERC20 tokens are mostly used for financial applications.	These are mostly used where validation of authentication is required. Ex – real state

OpenSea Platform



NFT



NFT contract

Things To Know

Listings: Listings refer to the NFTs available for sale at specified prices. In this case, there are no active listings under the "Listings" section, which means that no one has put this NFT up for sale at the moment.

Floor Difference : The floor price is the lowest price at which an NFT from a collection is listed for sale. The "Floor Difference" indicates how the current offers compare to this floor price

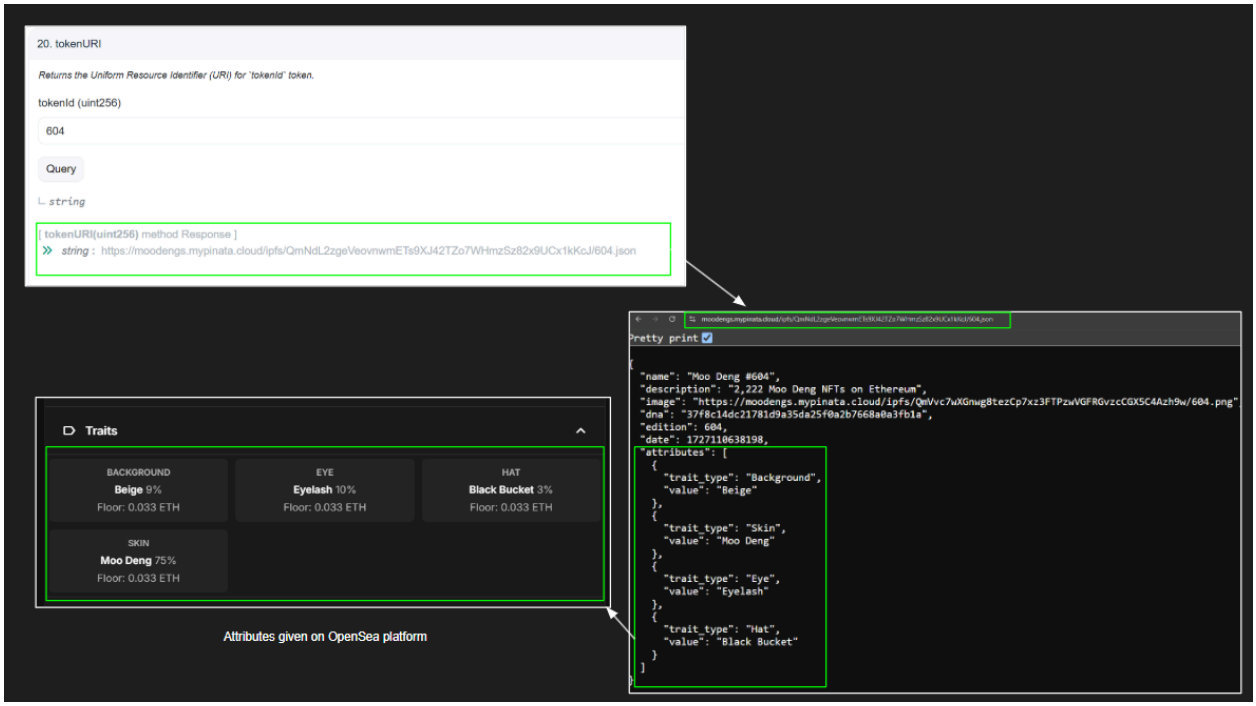
token URI

The URI (Uniform Resource Identifier) that points to the metadata for the NFT, such as images or descriptions stored off-chain (usually in decentralized storage systems like IPFS).

The Role of `tokenURI`:

- The `tokenURI` function is critical for wallets, marketplaces, and dApps to fetch the metadata of a specific token.
- This metadata is usually in JSON format and contains information like the token's name, description, image URL, and attributes. For example, for token ID 1, the data at the URI `https://my-nft-platform.com/metadata/1` might look like:

```
{
  "name": "Token 1",
  "description": "This is token number 1.",
  "image": "https://my-nft-platform.com/images/1.png",
  "attributes": [...]
}
```



Attributes given on OpenSea platform

Using Counters for Token IDs

using Counters for Counters.Counter: This tells Solidity to use the `Counters` library for managing a `Counters.Counter` data type.

Counters.Counter private _tokenIds: This creates a private instance of a counter, `_tokenIds`, which will be used to generate unique IDs for each new token (NFT) minted in this contract.

mint(player, newItemId);:

This line calls the _mint function from the ERC721 base contract, which creates a new token with ID newItemId and assigns it to the player address.

```
function _mint(address to, uint256 tokenId) internal {    - gas
    if (to == address(0)) {
        revert ERC721InvalidReceiver(address(0));
    }
    address previousOwner = _update(to, tokenId, address(0));
    if (previousOwner != address(0)) {
        revert ERC721InvalidSender(address(0));
    }
}
```

_setTokenURI(newItemId, tokenURI);:

This function sets the metadata URI for the newly minted token, associating the NFT with off-chain data (images, descriptions, etc.).

```
function _setTokenURI(uint256 tokenId, string memory _tokenURI) internal virtual {    - gas
    _tokenURIs[tokenId] = _tokenURI;
    emit MetadataUpdate(tokenId);
}
```

Supports Interface

```
function supportsInterface(bytes4 interfaceId) public view virtual override(ERC165, IERC165) returns (bool) {
    return
        interfaceId == type(IERC721).interfaceId ||
        interfaceId == type(IERC721Metadata).interfaceId ||
        super.supportsInterface(interfaceId);
}
```

ERC-165 Standard: This standard provides a way for smart contracts to declare the interfaces they implement. It defines the `supportsInterface(bytes4 interfaceId)` function, which returns `true` if the contract supports a particular interface and `false` otherwise. This standard helps with interoperability, allowing external entities to query the contract and understand which functionalities it supports, without needing to interact with the entire contract.

When an external contract or service interacts with your contract, it doesn't know beforehand which features are available. The `supportsInterface` function lets external entities check which features (i.e., interfaces) are implemented. For example:

- A marketplace might check if a contract supports the ERC-721 Metadata extension (`IERC721Metadata`) to know whether it can retrieve token metadata (like `name`, `symbol`, and `tokenURI`).

Supports Interface

The interface ID `0x80ac58cd` for `IERC721` is calculated using the **ERC-165 standard**. It is derived by XOR-ing the function selectors of all functions defined in the interface, excluding constructors and events.

Steps to Calculate the Interface ID:

1. **Function Selector:** The selector for a function is the first 4 bytes of the Keccak-256 hash of the function's signature (`functionName(parameterTypes)`).
2. **XOR the Selectors:** Take the XOR of the selectors of all functions in the interface.

Example for `IERC721`:

The `IERC721` interface contains these functions:

- `balanceOf(address)` → Selector: `0x70a08231`
- `ownerOf(uint256)` → Selector: `0x6352211e`
- `safeTransferFrom(address,address,uint256)` → Selector: `0x42842e0e`
- `transferFrom(address,address,uint256)` → Selector: `0x23b872dd`
- `approve(address,uint256)` → Selector: `0x095ea7b3`
- `setApprovalForAll(address,bool)` → Selector: `0xa22cb465`
- `getApproved(uint256)` → Selector: `0x081812fc`
- `isApprovedForAll(address,address)` → Selector: `0xe985e9c5`

IERC721 Interface ID calculation

0x70a08231 ^ 0x6352211e ^ 0x42842e0e ^ 0x23b872dd ^ 0x095ea7b3 ^ 0xa22cb465 ^ 0x081812fc ^ 0xe985e9c5 = 0x80ac58cd

baseURI

```
 */
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {    - gas
    _requireOwned(tokenId);

    string memory baseURI = _baseURI();
    return bytes(baseURI).length > 0 ? string.concat(baseURI, tokenId.toString()) : "";
}

/**
 * @dev Base URI for computing {tokenURI}. If set, the resulting URI for each
 * token will be the concatenation of the `baseURI` and the `tokenId`. Empty
 * by default, can be overridden in child contracts.
 */
function _baseURI() internal view virtual returns (string memory) {    - gas
    return "";
}
```

baseURI

This function generates the full URI for a specific token by calling `_baseURI()` and appending the `tokenId` to it.
The resulting URI is used to point to the metadata of the token, which typically includes information like the token's name, description, image, and other attributes.

Example: If the `baseURI` is `https://example.com/token/` and the `tokenId` is 123, then the resulting URI would be `https://example.com/token/123`.

Example Usage:

Let's say you are deploying a collection of NFTs (like artwork or collectibles), and you want each token's metadata to be hosted at a URL like:

- `https://my-nft-platform.com/metadata/1`
- `https://my-nft-platform.com/metadata/2`
- `https://my-nft-platform.com/metadata/3`

baseURI

Example Usage:

Let's say you are deploying a collection of NFTs (like artwork or collectibles), and you want each token's metadata to be hosted at a URL like:

- `https://my-nft-platform.com/metadata/1`
- `https://my-nft-platform.com/metadata/2`
- `https://my-nft-platform.com/metadata/3`

You would override the `_baseURI()` function in your contract like this:

```
function _baseURI() internal view virtual override returns (string memory) {

    return "https://my-nft-platform.com/metadata/";

}
```

🔹 What is Base URI & Token URI?

- **Base URI** → The **starting part** of the NFT metadata URL (e.g., `ipfs://Qm123.../`).
- **Token URI** → The **full metadata URL** for each NFT (e.g., `ipfs://Qm123.../1.json`).

✅ How is Token URI Generated?

1. **Base URI is set** when deploying the contract.
2. **Token ID is assigned** when minting an NFT.
3. **Token URI is formed** by combining Base URI + Token ID.

📌 **Example Process:**

- **Base URI** → `ipfs://QmXYZ123/`
- **Mint Token #1** → `ipfs://QmXYZ123/1.json`
- **Mint Token #2** → `ipfs://QmXYZ123/2.json`

🔹 Step-by-Step Process

1️⃣ Base URI is Set in Smart Contract

When deploying the smart contract, you define the **Base URI**.

✅ **How it Works:**

- `baseURI` is stored as `"ipfs://QmXYZ123/"`.
- When Bob mints **Token #1**, the contract creates `ipfs://QmXYZ123/1.json`.
- When Alice mints **Token #2**, the contract creates `ipfs://QmXYZ123/2.json`.

2️⃣ Where is Metadata Stored?

Each **Token URI** points to a metadata file (JSON) stored on:

- **IPFS** → Decentralized storage (e.g., `ipfs://QmXYZ123/1.json`).
- **Arweave** → Permanent web storage (e.g., `https://arweave.net/xyz...`).
- **Your Own Server** → Centralized storage (e.g., `https://yourwebsite.com/metadata/1.json`).

📌 **Example Metadata (`1.json`)**

```json
{
  "name": "CryptoTiger #1",
  "description": "A rare tiger NFT",
  "image": "ipfs://QmABC456/tiger.png",
  "attributes": [
    { "trait_type": "Background", "value": "Blue" },
    { "trait_type": "Rarity", "value": "Legendary" }
  ]
}
```

3️⃣ How Marketplaces Read Token URI?

Marketplaces like **OpenSea, Rarible, or Your Own NFT Marketplace**:

1. Read the **smart contract** on the blockchain.
2. Get the **Token URI** for each NFT.
3. Fetch metadata from **IPFS or a server**.
4. Display the NFT image and attributes.

📌 **How OpenSea Reads Token URI?**

- Calls `tokenURI(1)` on the contract.
- Gets `"ipfs://QmXYZ123/1.json"`.
- Fetches JSON metadata from **IPFS**.
- Displays the NFT image and attributes.

💡 Simple Example

Real-Life Example

- Think of Base URI like a **folder link:**

```arduino
https://myserver.com/nft-metadata/
```

- Each NFT gets a **file name:**

```arduino
https://myserver.com/nft-metadata/1.json
```

- OpenSea fetches this file & displays the NFT.

🚀 Final Answer

- **Blockchain tracks** → Ownership, Token ID, Token URI, and transaction history.
- **IPFS stores** → NFT metadata & images.
- **Marketplaces (OpenSea, Rarible) read the blockchain** to display NFTs.

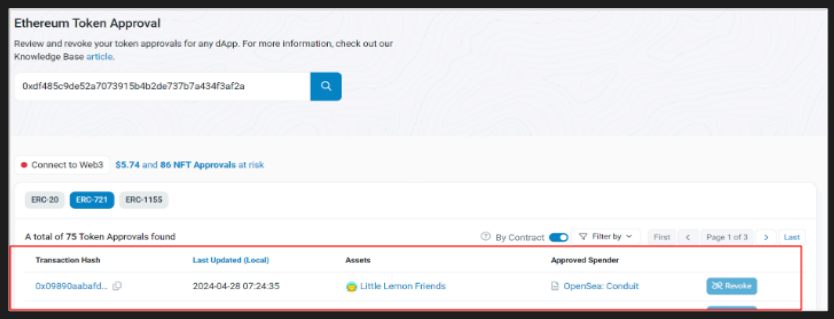
👉 **NFTs exist on the blockchain, but images & metadata are stored off-chain (IPFS).**

transferFrom vs safeTransferFrom

In ERC-721, transferFrom transfers NFTs without checking if the recipient can handle them, which can lead to tokens being lost if sent to incompatible contracts. In contrast, safeTransferFrom checks if the recipient address can handle ERC-721 tokens by calling onERC721Received on the recipient contract. If the recipient can't handle NFTs, safeTransferFrom reverts the transaction, preventing loss. Thus, safeTransferFrom is generally safer for transferring NFTs to contracts.

approve vs setApprovalForAll

In ERC-721, approve grants permission to one specific address to manage a single NFT (identified by its token ID). In contrast, setApprovalForAll authorizes an address to manage all NFTs owned by the caller, allowing bulk management. Use approve for single-item permissions and setApprovalForAll when you need to authorize an operator (like a marketplace) to manage multiple tokens on your behalf.



ERC721URIStorage

This is an extension of the ERC-721 standard (used for creating NFTs) that allows token metadata (like images or descriptions) to be stored on-chain via token URIs. The metadata is usually stored off-chain (e.g., IPFS) but referenced using a URI.

ERC721URIStorage

Token Metadata (URI) Storage: ERC721URIStorage stores a unique Uniform Resource Identifier (URI) for each NFT directly on-chain. This URI typically points to off-chain metadata (like JSON files), which contains information about the NFT such as: Name, Description, Image or other media

Per-Token URIs: Unlike the base ERC721 implementation where token URIs might be handled in a more global or uniform way, ERC721URIStorage lets you assign a different URI to each individual token. This makes it ideal for use cases where each NFT has unique characteristics or metadata.

Modifying the Token URI: With ERC721URIStorage, it's possible to change the URI associated with an NFT after it has been minted. This adds flexibility but also introduces some risks because changing metadata post-mint can affect the integrity or authenticity of the NFT.

Why Use ERC721URIStorage?

Easy Custom Metadata Handling: If you want each NFT to have unique metadata that can be referenced by an external link (e.g., to an IPFS-hosted JSON file), this extension provides that functionality.

On-chain Storage for URIs: It allows the URI to be stored and tracked directly on-chain, making it easier for external applications (like NFT marketplaces) to retrieve the token metadata.

🔹 How an NFT is Created & Tracked

1️⃣ Alice Deploys a Smart Contract

- Sets **Base URI** (e.g., `ipfs://QmXYZ123/`).
- Defines the **minting function** (rules for creating NFTs).

2️⃣ Bob Mints an NFT

- The contract **creates a new Token ID** (e.g., `#1`).
- It **generates the Token URI** by appending Token ID to the Base URI:

```java
Token URI = Base URI + Token ID
          = ipfs://QmXYZ123/1.json
```

- The blockchain stores:
  - Token ID = `1`
  - Owner = `0xBobWallet`
  - Token URI = `ipfs://QmXYZ123/1.json`

3️⃣ Token URI Points to IPFS (Metadata Storage)

- `ipfs://QmXYZ123/1.json` contains:

```json
{
  "name": "CryptoTiger #1",
  "description": "A rare NFT",
  "image": "ipfs://QmABC456/tiger.png",
  "attributes": [
    { "trait_type": "Background", "value": "Blue" },
    { "trait_type": "Rarity", "value": "Legendary" }
  ]
}
```

- The **image itself is also on IPFS**, linked inside the metadata.

4️⃣ Bob's NFT Appears on OpenSea

- OpenSea **reads the blockchain** and sees:
  - Bob owns Token #1.
  - Token URI → `ipfs://QmXYZ123/1.json`.
- OpenSea fetches metadata from IPFS and **displays the NFT**.

5️⃣ Bob Lists the NFT for Sale

- He puts Token #1 on sale in OpenSea.
- If **Charlie buys it**, the blockchain updates:
  - **New Owner** = `0xCharlieWallet`.
- OpenSea **automatically updates** the display.

🚀 Final Summary

✔️ **Base URI** is set when deploying the contract.
✔️ **Token URI** is generated when a user mints an NFT.
✔️ **Token URI is stored on the blockchain** (not the actual metadata).
✔️ **Token URI points to metadata on IPFS**, which contains the image and details.
✔️ **Marketplaces (like OpenSea) read the blockchain** to show NFTs.

👉 **That's how NFTs work!** 🎉

Let me know if you need any more clarifications! 😊🔥

📌 Final Summary

- ✅ **Base URI contains a link to off-chain storage** (IPFS, Arweave, etc.).
- ✅ **Token URI is generated** by appending Token ID to the Base URI.
- ✅ **Blockchain only stores the Token URI**, not the metadata or image.
- ✅ **Marketplaces like OpenSea read the Token URI to display the NFT.**

NFT Minter Contract

NFT minting contract prerequisites topics

- Merkle Tree & Merkle Proof
- Reentrancy Attack & Guard
- Payment Splitter
- tx.origin

📌 Why is Merkle Proof Used? (Simple Explanation)

🔹 **Merkle Proof is used for verifying if a person (or data) is part of a group without revealing the whole list.**

🔹 It is mainly used in **NFT pre-sales, whitelist systems, and blockchain verification**.

🎯 Real-Life Example: VIP Party Entry (Whitelist System)

Imagine a **VIP party** 🎉 where only selected guests can enter.

- **Alice is the party organizer.**
- She has a **guest list** with 1,000 names.
- **Bob wants to enter the party**, but Alice doesn't want to show the full guest list.

💡 **Solution: Use Merkle Proof!**

1️⃣ **Alice creates a "Merkle Tree"** 🛠️

- She combines all guest names into a special structure (Merkle Tree).
- The **final "Merkle Root"** (a single unique code) is stored in the system.

2️⃣ **Bob proves he is on the list using Merkle Proof** 📝

- Instead of showing the full list, Bob provides a **Merkle Proof** (a few pieces of the tree).
- The system **checks the Merkle Root** and verifies Bob is on the list.

3️⃣ **Bob gets entry without seeing the full guest list!** ✅

📌 How is This Used in NFT Whitelisting?

In NFT pre-sales:

✔ The "**guest list**" = **whitelist addresses**.

✔ The "**VIP party**" = **NFT minting event**.

✔ Users **prove they are on the list** using Merkle Proof.

✔ The contract **verifies without revealing the entire list**.

🚀 **This saves gas fees & keeps data private!**

📝 Final Summary:

- **Merkle Proof** helps verify if someone is in a list **without revealing the full list**.
- Saves gas fees, **keeps data small, fast, and private**.
- Used for **NFT whitelist systems, blockchain verification, and security**.

Merkle-Tree Generator

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/utils/cryptography/MerkleProof.sol";

contract MerkleTree {

    bytes32 private root;

    constructor(bytes32 _root) {
        root = _root;
    }

    function safeMint(bytes32[] memory proof) public view returns(bool) {
        require(isValid(proof, keccak256(abi.encodePacked(msg.sender))),
"Not a part of Allowlist");
        return true;
    }
```

```
    function isValid(bytes32[] memory proof, bytes32 leaf) internal view
returns (bool) {
        return MerkleProof.verify(proof, root, leaf);
    }
}
```

```
const { MerkleTree } = require("merkletreejs");
const keccak256 = require("keccak256");

// Step 2: Define the allowlist and create a Merkle Tree
const allowlistAddresses = [
    '0x5B38Da6a701c568545dCfcB03FcB875f56beddC4',
    '0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2',
    '0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db',
    '0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB'
];

const leafNodes = allowlistAddresses.map(addr => keccak256(addr));
const merkleTree = new MerkleTree(leafNodes, keccak256, { sortPairs: true
});
const root = merkleTree.getHexRoot();

// Step 5: Generate the Merkle proof for a specific address
const addressToVerify = "0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2";
const leaf = keccak256(addressToVerify);
const proof = merkleTree.getHexProof(leaf);
// Convert to JSON string and remove all whitespace
const proofString = JSON.stringify(proof).replace(/\s+/g, '');

console.log("Proof String:", proofString);
console.log("Root:", root)
```

How Do Whitelisted Users Know They Need to Generate Proof?

Whitelisted users don't **manually** generate proof themselves. The **DApp (minting website)** does it automatically for them!

Step-by-Step Explanation

1️⃣ **Alice (Contract Owner) Creates & Shares the Whitelist**

- She **publishes the whitelist** (e.g., on a website, Discord, or Twitter).
- She also **deploys the smart contract** with the **Merkle Root** stored inside.

2️⃣ **Bob (Whitelisted User) Visits the Minting Website**

- Bob **goes to the official NFT minting DApp** (a website built by Alice).
- He **connects his wallet** (MetaMask, WalletConnect, etc.).

3️⃣ **The Website Automatically Checks Bob's Address**

- The website **checks if Bob is on the whitelist** (it has the original 1,000 addresses).
- If **Bob is in the whitelist**, the website **automatically generates the Merkle Proof for him**.

4️⃣ **Bob Clicks "Mint" → Proof is Sent to the Contract**

- The website **sends Bob's proof** to the smart contract.
- The **contract verifies the proof**.
- If valid → ✅ **Bob is allowed to mint**.
- If invalid → ❌ **Minting fails**.

Do Whitelisted Users Need to Manually Generate Proof?

No! The website **automatically generates & submits the proof** when they try to mint.

- 🔹 **Users just need to visit the minting site and connect their wallet.**
- 🔹 The **DApp does all the proof generation & verification behind the scenes.**

Example in Simple Words (Real Life)

Imagine a **VIP party** 🎉:

- Alice creates a **guest list** of 1,000 people.
- When Bob arrives at the **VIP entrance**, he **doesn't need to bring the full list**.
- The **bouncer (minting website) checks Bob's name** in the system.
- If Bob is **on the list, he gets a special token (Merkle Proof) to enter**.
- If Bob **is not on the list, he cannot enter**.

Final Answer:

✅ Whitelisted users **just visit the minting website & connect their wallet**.
✅ The website **automatically generates & submits the proof** for them.
✅ They **don't need to manually generate or check anything**!

📌 What is a CID in IPFS?

CID (Content Identifier) is a unique identifier used in **IPFS (InterPlanetary File System)** to locate and retrieve files. Instead of a traditional URL, CID is a **hash-based** address that points to a specific file stored on IPFS.

🔹 How CID Works?

1️⃣ Bob uploads an NFT image (`tiger.png`) to IPFS.

- IPFS generates a unique **CID** based on the file content.
- Example CID:

```nginx
QmABC123xyz789
```

- This CID is like a fingerprint of the file.

2️⃣ Bob also uploads metadata (`1.json`) containing NFT details.

- Metadata file:

```json
{
  "name": "CryptoTiger #1",
  "description": "A rare tiger NFT",
  "image": "ipfs://QmABC123xyz789"
}
```

- **CID for metadata:**

```nginx
QmXYZ456def987
```

3️⃣ NFT Contract Uses the Metadata CID

- When minting, the contract stores:

```java
Token URI = ipfs://QmXYZ456def987
```

- Marketplaces like OpenSea fetch the metadata from this **CID**.

🔹 Key Features of CID

✅ **Immutable** – Once a CID is generated, it never changes.
✅ **Content-Based** – If a file changes, it gets a new CID.
✅ **Decentralized** – Works on IPFS, no single server dependency.

🔹 Example in NFT Storage

Type	Example CID	Usage
NFT Image	`QmABC123xyz789`	Points to `tiger.png`
Metadata	`QmXYZ456def987`	Points to `1.json` (Token URI)

📌 **Final Note:** CID is like a **permanent address for a file on IPFS**. If you store NFT metadata or images on IPFS, their CID is used in the blockchain to reference them.

how based on cid find the data in decentrelization?

🔍 How Does Decentralized Storage Retrieve Data Using a CID?

When you store a file on **IPFS (InterPlanetary File System)**, it gets a **CID (Content Identifier)**. To retrieve the file, you don't need a traditional URL; instead, you use the CID. But how does IPFS find the file in a decentralized way? Let's break it down!

🔹 Step-by-Step: Retrieving Data Using CID

1️⃣ Bob Uploads an NFT Image to IPFS

- Bob uploads `tiger.png` to IPFS.
- IPFS generates a unique **CID** based on the file content.
- Example CID:

```nginx
QmABC123xyz789
```

- The file is stored in IPFS nodes.

2️⃣ Nodes Store & Share the File

- The file is **broken into smaller pieces (chunks)**.
- Multiple IPFS nodes **store copies** of the file.
- The CID is like a **fingerprint** that points to the correct data.

3️⃣ David Wants to Download the NFT

- He requests the file using:

```cpp
ipfs://QmABC123xyz789
```

- IPFS looks for the **closest nodes** that have this file.

4️⃣ IPFS Uses a Distributed Network (DHT)

- **DHT (Distributed Hash Table)** helps find the nearest node with the CID.
- The system **routes** the request to the right node.
- The file **reassembles from chunks** and gets downloaded.

5️⃣ David Gets the NFT Image

- The original **file remains unchanged**, ensuring authenticity.
- Decentralized! No central server needed.

🔹 How is This Different from a Traditional URL?

Traditional URL (Centralized)	CID (IPFS, Decentralized)
Points to a **location** (e.g., `https://example.com/image.png`)	Points to the **content itself** (e.g., `ipfs://QmABC123xyz789`)
Server dependent – If the website goes down, the file is lost	**Node-based** – Anyone can store and retrieve the file
Can be **changed** or **deleted** by the owner	**Permanent & Immutable** (CID changes only if the file changes)

🔹 Real-Life Example (Decentralized Network)

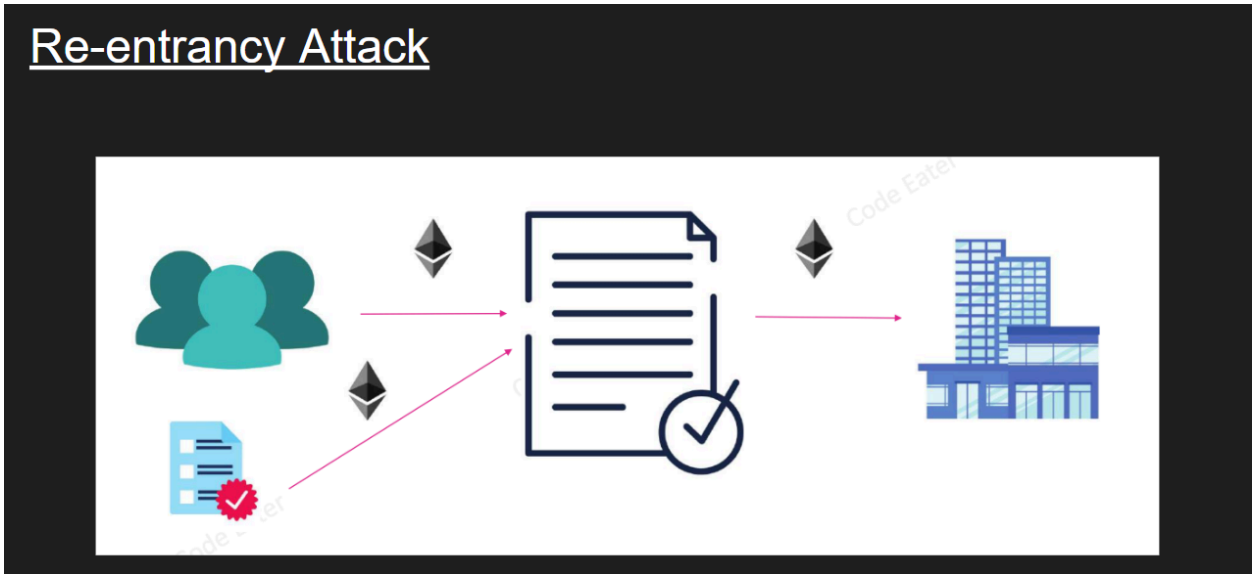
💡 Imagine **IPFS as a giant decentralized library** 📚

- Each book (file) has a **unique fingerprint (CID)** instead of a fixed shelf location.
- You ask other librarians (nodes) for the book using its **fingerprint (CID)**.
- The library finds the book (data) **from the nearest location**.

🚀 That's how IPFS finds and retrieves data without a central server! 🔥

📌 Summary

✅ IPFS **doesn't store data on a central server**; it distributes it across multiple nodes.
✅ The **CID uniquely identifies content**, not a location.
✅ The **DHT network** helps find the nearest node storing the data.
✅ IPFS ensures **permanent, decentralized storage**, unlike traditional URLs.



🔍 Reentrancy Attack: Explained Simply

A **Reentrancy Attack** is a **smart contract vulnerability** where a malicious contract repeatedly calls a function **before the previous call is completed**, draining funds.

🔹 How Does a Reentrancy Attack Work?

1️⃣ **A smart contract (Bank)** allows users to **deposit and withdraw** Ether (ETH).
2️⃣ The contract **sends funds before updating balances**.
3️⃣ A **malicious contract** exploits this by repeatedly calling `withdraw()` **before the balance updates**.
4️⃣ The attacker **keeps draining** funds until the contract runs out of ETH.

🔹 Step-by-Step Breakdown (With Diagram)

🔸 Normal Scenario (No Attack)

1️⃣ Bob deposits 10 ETH

- `balances[Bob] = 10`

  2️⃣ Bob calls `withdraw()`

- The contract **sends 10 ETH**
- ✅ **Updates balance** (`balances[Bob] = 0`)

🟢 Everything is **safe** ✅

🔸 Reentrancy Attack Scenario

1️⃣ Attacker deposits 1 ETH

- `balances[Attacker] = 1`

2️⃣ Attacker **calls** `withdraw()`

- The contract **sends 1 ETH**
- 🚨 **Before balance is updated**, the attacker calls `withdraw()` AGAIN
- The contract **sends another 1 ETH**
- Attack repeats **until the contract is drained**

🔴 The Problem: Why Does the System Fail?

Let's assume:

- You have **10 ETH** in your balance.
- You **withdraw 1 ETH** at a time.
- But the system **forgets** to update your balance **before** sending the money.

Step-by-Step Breakdown of the Attack

1️⃣ **User Calls `requestEther()` to Withdraw 1 ETH**

- The smart contract checks `balance[msg.sender]` → **it's 10 ETH**
- The contract **sends 1 ETH** first 🚀

2️⃣ **Before the Balance is Updated, the Attacker Calls the Function Again**

- The contract **still thinks the balance is 10 ETH** ❌ (because it hasn't been updated yet)
- So the attacker **withdraws 1 ETH again**

3️⃣ **Repeat This Trick Multiple Times Before Updating Balance**

- The attacker **keeps calling the function**
- The contract **keeps sending money** (thinking there's still 10 ETH)
- Eventually, the contract **loses all its money** 😱

💡 Why Does This Happen?

The problem is that **the contract updates the balance too late.**

- It should **first set balance to 0**
- Then send the money

Bad Code (Vulnerable to Attack)

```solidity
function requestEther() public {
    (msg.sender).transfer(balance[msg.sender]);  // ❌ Sends money first
    balance[msg.sender] = 0;  // ❌ Updates balance too late
}
```

🚨 **Problem:** The attacker **re-enters** the function before the balance is updated.

Good Code (Safe from Attack)

```solidity
function requestEther() public {
    uint256 amount = balance[msg.sender];
    balance[msg.sender] = 0;  // ✅ Updates balance first
    payable(msg.sender).transfer(amount);  // ✅ Then sends money
}
```

✅ **Now, even if the attacker tries to re-enter, their balance is already 0**, so they **can't keep withdrawing**.

🌍 Real-Life Example: ATM Problem

Bad ATM (Vulnerable to Attack)

- You have **₹10,000** in your bank.
- You **withdraw ₹1,000**, but the bank **forgets to update your balance first**.
- Before the balance is updated, you quickly **withdraw another ₹1,000**.
- You keep repeating this until the ATM is empty! 😱

Good ATM (Secure)

- As soon as you **withdraw ₹1,000**, the bank **updates your balance first**.
- Now, if you try to withdraw again, the system **knows your new balance**.

🔹 How to Prevent Reentrancy Attacks?

✅ Update balance before sending funds

```solidity
function withdraw() public {
    uint256 amount = balances[msg.sender];

    // ✅ Update balance first!
    balances[msg.sender] = 0;

    (bool success, ) = msg.sender.call{value: amount}("");
    require(success, "Transfer failed");
}
```

✅ Use Reentrancy Guard

```solidity
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract SafeBank is ReentrancyGuard {
    function withdraw() public nonReentrant {
        uint256 amount = balances[msg.sender];
        balances[msg.sender] = 0;
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");
    }
}
```

✅ Use `transfer()` or `send()` instead of `call{value: amount}("")`

- `call{value: amount}("")` forwards all gas (risk of reentrancy).
- `transfer()` and `send()` only forward 2300 gas (safe).

so balace 0 is just like the key am i right?

Yes! **Setting** `balance[msg.sender] = 0` **first is like a security key.** 🔑

It **does not delete** the user's remaining balance. Instead, it **locks** the current withdrawal to prevent re-entrancy attacks (hacking).

Think of it like this:

🛒 Real-Life Example: Online Shopping Refund

1️⃣ You request a **₹1,000 refund** from an online wallet.
2️⃣ The system **locks your refund request first** (sets balance to ₹0 temporarily).
3️⃣ Then, the system **processes the refund** and sends ₹1,000 to you.
4️⃣ Your wallet is updated correctly, and any remaining balance **is still safe.**

🔹 Why Set `balance[msg.sender] = 0` First?

- **Without setting it first**, a hacker can call the function **multiple times** before the system updates the balance.
- **By setting it first**, we **lock the transaction** and ensure only one withdrawal happens at a time.

✅ **So, balance = 0 is like a security lock 🔒 that prevents multiple withdrawals.**

Does this explanation help? 😊

Re-entrancy Attack

```
receive external payable{
requestEther();
}
```

```
function requestEther(){
(msg.sender).transfer(balance[msg.sender]);
balance[msg.sender]=0;
}
```

Code Eater

🚨 The Bank Teller Example (Vulnerable System)

Imagine you go to a bank and ask for ₹10,000 from your account.

- The **bank teller** checks your balance (₹10,000) and **hands over your cash first**.
- Before the teller updates the system to ₹0, you **quickly ask for ₹10,000 again**!
- The teller **sees the same balance (₹10,000)** and hands you money again!

❌ **This is a re-entrancy attack!** You tricked the system into giving you money multiple times before it updates the balance.

✅ Safe Bank Teller (Using a Lock)

Now, the bank installs a **new security system**:

1️⃣ **Before giving money, the teller updates your balance to ₹0**.

2️⃣ Then they give you your ₹10,000.

3️⃣ If you try to ask for money again, the teller sees your balance is already **₹0**, so they **stop the transaction**.

Re-entrancy Attack

```
function withdraw() public{

    uint bal = balances[msg.sender];
    require(bal > 0);// eth >0

    (bool sent, ) = msg.sender.call{value: bal}("");
    require(sent, "Failed to send Ether");

    balances[msg.sender] = 0;

}
```

Re-entrancy Attack

```
function withdraw() public{

    uint bal = balances[msg.sender];
    require(bal > 0);// eth >0

    (bool sent, ) = msg.sender.call{value: bal}("");
    require(sent, "Failed to send Ether");

    balances[msg.sender] = 0;

}
```

Re-entrancy Attack

```
function withdraw() public{

    uint bal = balances[msg.sender];
    require(bal > 0);// eth >0

    (bool sent, ) = msg.sender.call{value: bal}("");
    require(sent, "Failed to send Ether");

    balances[msg.sender] = 0;

}
```

Re-entrancy Attack

```
bool lock=false;

        function withdraw() public{
            require(lock==false,”Locked”);

            uint bal = balances[msg.sender];
            require(bal > 0);// eth >0

            (bool sent, ) = msg.sender.call{value: bal}("");
            require(sent, "Failed to send Ether");

            balances[msg.sender] = 0;

            lock=false;
        }
```

^^^^above slide is false code

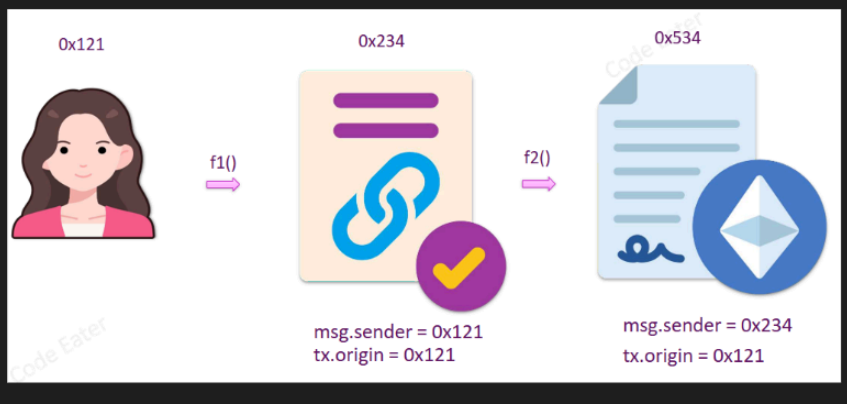
✅ Corrected Code (Fixing the Issue)

solidity Copy Edit

```solidity
bool lock = false;

function withdraw() public {
    require(lock == false, "Locked");  // ✅ Check if function is already in use

    lock = true;  // ✅ Lock the function to prevent re-entrancy

    uint bal = balances[msg.sender];
    require(bal > 0);

    (bool sent, ) = msg.sender.call{value: bal}("");
    require(sent, "Failed to send Ether");

    balances[msg.sender] = 0;  // ✅ Update balance after sending Ether

    lock = false;  // ✅ Unlock after completing execution
}
```

Tx.Origin

tx.origin is a global variable in Ethereum smart contracts that represents the external account that initiated the transaction. It contains the address of the user account that first created and sent the transaction to the network.




```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract A {
    B public bContract;

    constructor(address _bAddress) {
        bContract = B(_bAddress);
    }

    function callB() public view returns (address, address) {
        return bContract.callC();
    }
}


contract B {
    C public cContract;

    constructor(address _cAddress) {
        cContract = C(_cAddress);
    }

    function callC() public view returns (address, address) {
```

```
        return cContract.getOriginAndSender();
    }
}


contract C {
    function getOriginAndSender() public view returns (address, address) {
        return (tx.origin, msg.sender);
    }
}
```

Payment Splitter

```
import "@openzeppelin/contracts/finance/PaymentSplitter.sol";

contract SplitPayments is PaymentSplitter {
    constructor (    infinite gas 1035600 gas
        address[] memory payees,
        uint256[] memory shares_
    ) payable PaymentSplitter(payees, shares_) {

    }
}
```

IPFS & Pinata

IPFS (InterPlanetary File System) is a decentralized file storage and sharing protocol. Instead of relying on centralized servers, IPFS uses a peer-to-peer network to store and retrieve files. Each file is assigned a unique cryptographic hash (CID) based on its content, ensuring immutability and enabling efficient retrieval. It's commonly used for storing large, static content like images, documents, and metadata.

Pinata is a service built on top of IPFS that simplifies file uploading, management, and access. It provides a user-friendly interface and tools for storing files on IPFS without needing to set up your own node. Pinata also offers additional features like pinning files (ensuring their persistence across the IPFS network), analytics, and content delivery optimization.

Together, IPFS ensures decentralized storage, and Pinata makes it easy to use and manage files on this system, commonly utilized in blockchain applications like NFTs to store metadata and media.

🔹 What This Contract Does

This contract lets users **mint NFTs** (Non-Fungible Tokens) by either:

1. **Presale (Whitelist Minting)**
   - Only whitelisted users can mint during this phase.
   - Each user has a limit (`PRESALE_LIMIT`).
   - They must prove they are whitelisted using **Merkle Proof**.
2. **Public Sale**
   - Open to everyone.
   - People can mint if they pay the correct amount.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "@openzeppelin/contracts/finance/PaymentSplitter.sol";
import "@openzeppelin/contracts/utils/cryptography/MerkleProof.sol";

contract NftMinting is ERC721, ReentrancyGuard, PaymentSplitter, Ownable {
    bytes32 immutable merkleRoot;
    string private baseURI;

    bool public isPaused;
    bool public isPreSaleActive;
    bool public isPublicSaleActive;

    uint256 currentTokenId;

    uint256 public constant PRESALE_LIMIT = 5;
    uint256 public constant NFT_MINTING_PRICE = 0.01 ether;
    uint256 public constant MAX_SUPPLY = 20;

    address[] private teamMembers = [
```

```
        0x5B38Da6a701c568545dCfcB03FcB875f56beddC4,
        0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2,
        0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db
    ];
    uint256[] private teamShares = [20, 40, 40];

    mapping(address => uint256) public presaleCount;
    mapping(uint256 => string) public tokenCids;

    modifier onlyEOA() {
        require(tx.origin == msg.sender, "Contract calls not allowed");
        _;
    }

    modifier isVerified(bytes32[] calldata proof) {
        require(
            MerkleProof.verify(
                proof,
                merkleRoot,
                keccak256(abi.encodePacked((msg.sender)))
            ),
            "Proof not valid"
        );
        _;
    }

    constructor(string memory initialBaseURI, bytes32 root)
        ERC721("NEW NFT", "NNT")
        Ownable(msg.sender)
        ReentrancyGuard()
        PaymentSplitter(teamMembers, teamShares)
    {
        baseURI = initialBaseURI;
        merkleRoot = root;
    }

    function togglePreSale() external onlyOwner {
        isPreSaleActive = !isPreSaleActive;
    }
```

```
    function togglePublicSale() external onlyOwner {
        isPublicSaleActive = !isPublicSaleActive;
    }

    function preSaleMint(
        uint256 nftAmount,
        bytes32[] calldata proof,
        string[] calldata cids
    ) external payable onlyEOA nonReentrant isVerified(proof) {
        require(isPublicSaleActive == true, "Contract not active");
        require(isPreSaleActive == true, "Presale not active");
        require(nftAmount > 0, "nftAmount > 0");
        require(
            presaleCount[msg.sender] + nftAmount <= PRESALE_LIMIT,
            "PRESALE_LIMIT exceed"
        );
        require(currentTokenId + nftAmount <= MAX_SUPPLY, "MAX_SUPPLY
exceed");
        require(cids.length == nftAmount, "Contract not active");
        require(
            msg.value == nftAmount * NFT_MINTING_PRICE,
            "Not enough ethers"
        );

        presaleCount[msg.sender] += nftAmount;

        for (uint256 i = 0; i < nftAmount; i++) {
            _mintToken(msg.sender, cids[i]);
        }
    }

    function _mintToken(address to, string calldata cid) internal {
        _safeMint(to, currentTokenId);
        tokenCids[currentTokenId] = cid;
        currentTokenId++;
    }

    function publicSaleMint(uint256 nftAmount, string[] calldata cids)
        external
        payable
```

```
        onlyEOA
        nonReentrant
    {
        require(!isPaused, "Contract: Paused");
        require(isPublicSaleActive, "Public sale is not active");
        require(nftAmount > 0, "Amount must be greater than zero");
        require(
            currentTokenId + nftAmount <= MAX_SUPPLY,
            "Max supply exceeded"
        );
        require(cids.length == nftAmount, "CIDs and nftAmount
mismatched");
        require(
            msg.value == nftAmount * NFT_MINTING_PRICE,
            "Not enough ethers"
        );

        for (uint256 i = 0; i < nftAmount; i++) {
            _mintToken(msg.sender, cids[i]);
        }
    }

    function setCid(uint256 tokenId, string calldata cid) external {
        require(msg.sender == _requireOwned(tokenId));
        tokenCids[tokenId] = cid;
    }

    function totalSupply() external view returns (uint256) {
        return currentTokenId;
    }

    function remainingSupply() external view returns (uint256) {
        return MAX_SUPPLY - currentTokenId;
    }

    function withdrawEther() external payable onlyOwner {
        require(address(this).balance > 0, "Contract has zero balance");
        payable(msg.sender).transfer(address(this).balance);
    }
}
```

```
//["bafkreibmzyvd4fcwdxsrydkpjq7cdg7o3nslpuxmq3fwfstbpnuk3qqclq","QmQNuYc7xXY62v6jVfFz2NoZMWZzeejTTB3Ke223aS7DqD"]
```

1 Importing Necessary Libraries

solidity Copy Edit

```solidity
import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "@openzeppelin/contracts/finance/PaymentSplitter.sol";
import "@openzeppelin/contracts/utils/cryptography/MerkleProof.sol";
```

- **ERC721.sol** → Allows the contract to create NFTs.
- **Ownable.sol** → Allows only the owner to perform certain actions.
- **ReentrancyGuard.sol** → Prevents re-entrancy attacks (security feature).
- **PaymentSplitter.sol** → Helps in splitting payments among multiple team members.
- **MerkleProof.sol** → Used to verify if a user is whitelisted (presale).

---

2 Contract Declaration

solidity Copy Edit

```solidity
contract NftMinting is ERC721,ReentrancyGuard,PaymentSplitter,Ownable{
```

- The contract is named `NftMinting`.
- It inherits functionalities from other contracts:
  - **ERC721** → NFT standard.
  - **ReentrancyGuard** → Prevents re-entrancy attacks.
  - **PaymentSplitter** → Handles team payment distribution.
  - **Ownable** → Gives ownership control.

3 Declaring Variables

```solidity
bytes32 immutable merkleRoot;
string private baseURI;
bool public isPaused;
bool public isPreSaleActive;
bool public isPublicSaleActive;
uint currentTokenId;
uint public constant PRESALE_LIMIT = 5;
uint public constant NFT_MINTING_PRICE = 0.01 ether;
uint public constant MAX_SUPPLY = 20;
```

- `merkleRoot` → Stores the Merkle Root (used for whitelist verification).
- `baseURI` → Stores the base URL for NFT metadata.
- `isPaused` → If `true`, the contract is paused (no minting allowed).
- `isPreSaleActive` → If `true`, presale is active.
- `isPublicSaleActive` → If `true`, public sale is active.
- `currentTokenId` → Tracks the total number of minted NFTs.
- `PRESALE_LIMIT = 5` → A user can only mint 5 NFTs in presale.
- `NFT_MINTING_PRICE = 0.01 ether` → Price of 1 NFT.
- `MAX_SUPPLY = 20` → Maximum 20 NFTs can be minted.

---

4 Defining Team Members and Their Share

```solidity
address[] private teamMembers = [
    0x5B38Da6a701c568545dCfcB03FcB875f56beddC4,
    0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2,
    0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db
];
uint256[] private teamShares = [20,40,40];
```

- `teamMembers` → List of 3 Ethereum wallet addresses receiving funds.
- `teamShares` → Distribution of the contract's earnings:
  - 1st member gets 20%,
  - 2nd member gets 40%,
  - 3rd member gets 40%.

5 Presale Minting Limit (Mapping)

```solidity
mapping(address => uint) public presaleCount;
```

- Tracks how many NFTs each address has minted during presale.

---

6 Storing NFT Metadata (CID)

```solidity
mapping(uint => string) public tokenCids;
```

- Each NFT token ID is linked to a CID (Content Identifier from IPFS).

---

7 Only EOA Modifier

```solidity
modifier onlyEOA(){
    require(tx.origin == msg.sender, "Contract calls not allowed");
    _;
}
```

- Prevents smart contracts from calling this contract (only real users can mint).

---

8 Whitelist Verification Modifier

```solidity
modifier isVerified(bytes32[] calldata proof){
    require(MerkleProof.verify(proof, merkleRoot, keccak256(abi.encodePacked((msg.sender)))),
    _;
}
```

- Checks if the user is whitelisted before allowing presale minting.

9️⃣ Contract Constructor

solidity Copy Edit

```solidity
constructor(string memory initialBaseURI, bytes32 root)
ERC721("NEW NFT","NNT")
Ownable(msg.sender)
ReentrancyGuard()
PaymentSplitter(teamMembers, teamShares){
    baseURI = initialBaseURI;
    merkleRoot = root;
}
```

- Sets up contract metadata:
  - NFT Collection Name = `"NEW NFT"`
  - Symbol = `"NNT"`
- Stores Merkle Root for presale whitelist.

🔟 Toggle Functions (Start/Stop Sales)

solidity Copy Edit

```solidity
function togglePreSale() external onlyOwner {
    isPreSaleActive = !isPreSaleActive;
}

function togglePublicSale() external onlyOwner{
    isPublicSaleActive = !isPublicSaleActive;
}
```

- Allows the owner to turn presale/public sale ON or OFF.

1️⃣ 1️⃣ Presale Minting Function

```solidity
function preSaleMint(uint nftAmount, bytes32[] calldata proof, string[] calldata cids)
external payable onlyEOA nonReentrant isVerified(proof){
```

- `onlyEOA` → Blocks bots/contracts.
- `nonReentrant` → Protects against re-entrancy attacks.
- `isVerified(proof)` → Checks whitelist status.

```solidity
require(isPublicSaleActive == true, "Contract not active");
require(isPreSaleActive == true, "Presale not active");
require(nftAmount > 0, "nftAmount > 0");
require(presaleCount[msg.sender] + nftAmount <= PRESALE_LIMIT, "PRESALE_LIMIT exceed");
require(currentTokenId + nftAmount <= MAX_SUPPLY, "MAX_SUPPLY exceed");
require(cids.length == nftAmount, "Contract not active");
require(msg.value == nftAmount * NFT_MINTING_PRICE, "Not enough ethers");
```

- Checks all conditions before minting.

```solidity
presaleCount[msg.sender] += nftAmount;
for(uint i = 0; i < nftAmount; i++){
    _mintToken(msg.sender, cids[i]);
}
```

- Increases presale count and mints NFTs.

1 2 NFT Minting Function

```solidity
function _mintToken(address to, string calldata cid) internal {
    _safeMint(to, currentTokenId);
    tokenCids[currentTokenId] = cid;
    currentTokenId++;
}
```

- Mints a new NFT and stores its CID.

---

1 3 Public Sale Minting

Similar to presale minting but doesn't require whitelist verification.

---

1 4 Set NFT Metadata

```solidity
function setCid(uint tokenId, string calldata cid) external {
    require(msg.sender == _requireOwned(tokenId));
    tokenCids[tokenId] = cid;
}
```

- Allows NFT owner to update metadata CID.

---

1 5 Withdraw Ether

```solidity
function withdrawEther() external payable onlyOwner {
    require(address(this).balance > 0, "Contract has zero balance");
    payable(msg.sender).transfer(address(this).balance);
}
```

- Transfers all contract funds to the owner.

What is `nftAmount`?

- `nftAmount` is the **number of NFTs** that the user wants to mint (buy).
- When calling the `preSaleMint` or `publicSaleMint` function, the user provides `nftAmount` as an input.
- For example, if a user wants to mint **3 NFTs**, they pass `nftAmount = 3`.

Condition 1:

```solidity
require(nftAmount > 0, "nftAmount > 0");
```

- **Checks if the user is minting at least 1 NFT.**
- **Why?** Because minting **0** NFTs makes no sense.
- If the user enters `nftAmount = 0`, the transaction fails.

Condition 2:

```solidity
require(presaleCount[msg.sender] + nftAmount <= PRESALE_LIMIT, "PRESALE_LIMIT exceed");
```

- **Ensures that the user does not mint more than the allowed presale limit.**
- The presale limit (`PRESALE_LIMIT`) is **5**.
- `presaleCount[msg.sender]` keeps track of how many NFTs this user has already minted during the presale.
- If the user already minted **3 NFTs** and now wants to mint **3 more**, the total would be **6**, which **exceeds the presale limit of 5**, so the transaction fails.

Condition 3:

```solidity
require(currentTokenId + nftAmount <= MAX_SUPPLY, "MAX_SUPPLY exceed");
```

- **Ensures that we don't mint more NFTs than the maximum supply.**
- `MAX_SUPPLY = 20`, meaning only **20 NFTs** can ever exist.
- `currentTokenId` keeps track of how many NFTs have already been minted.
- If **15 NFTs** have already been minted and the user tries to mint **6 more**, the total would be **21**, exceeding `MAX_SUPPLY`. So, the transaction fails.

Condition 4:

```solidity
require(cids.length == nftAmount, "Contract not active");
```

- **Ensures that the number of NFTs and the number of provided CIDs match.**
- `cids` is an array that contains unique **content identifiers (CIDs)** for each NFT.
- If a user wants to mint **3 NFTs**, they should provide **3 CIDs**.
- If they send only **2 CIDs**, this condition fails because `cids.length (2) ≠ nftAmount (3)`.

Condition 5:

```solidity
require(msg.value == nftAmount * NFT_MINTING_PRICE, "Not enough ethers");
```

- **Ensures that the user has sent enough Ether to pay for the NFTs.**
- `NFT_MINTING_PRICE = 0.01 ether`, meaning each NFT costs **0.01 Ether**.
- If the user wants to mint **3 NFTs**, they must send:

  ```
  3 * 0.01 ether = 0.03 ether
  ```

- If they send **less than 0.03 ether**, the transaction fails.